1. **Create two threads: one prints even numbers, another prints odd numbers (1–100) in sequence.**
   Code:

```
package Day9;
class EvenOddPrinter {
    private int number = 1;
    private final int MAX = 100;

    public synchronized void printOdd() {
        while (number <= MAX) {
            if (number % 2 != 0) {
                System.out.println("Odd: " + number);
                number++;
                notify();
            } else {
                try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
            }
        }
    }

    public synchronized void printEven() {
        while (number <= MAX) {
            if (number % 2 == 0) {
                System.out.println("Even: " + number);
                number++;
                notify();
            } else {
                try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
            }
        }
    }
}

public class AlternatePrinting {
    public static void main(String[] args) {
        EvenOddPrinter printer = new EvenOddPrinter();

        Thread oddThread = new Thread(printer::printOdd);
        Thread evenThread = new Thread(printer::printEven);

        oddThread.start();
        evenThread.start();
    }
}
```

2. **Three threads print A, B, C repeatedly in order (ABCABC…).**
   Code:

```
package Day9;

class PrintABC {
    private int state = 0;

    public synchronized void printA() {
        while (true) {
            while (state % 3 != 0) {
                try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
            }
            System.out.print("A ");
            state++;
            notifyAll();
        }
    }

    public synchronized void printB() {
        while (true) {
            while (state % 3 != 1) {
                try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
            }
            System.out.print("B ");
            state++;
            notifyAll();
        }
    }

    public synchronized void printC() {
        while (true) {
            while (state % 3 != 2) {
                try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
            }
            System.out.print("C ");
            state++;
            notifyAll();
        }
    }
}

public class PrintABCSequence {
    public static void main(String[] args) {
        PrintABC printer = new PrintABC();
```

```
        new Thread(printer::printA).start();
        new Thread(printer::printB).start();
        new Thread(printer::printC).start();
    }
}
```

3. **Multiple threads deposit and withdraw from same account safely.**
   Code:

```
package Day9;

class BankAccount {
    private int balance = 1000;

    public synchronized void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount + " | Balance: " + balance);
    }

    public synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println("Withdrew: " + amount + " | Balance: " + balance);
        } else {
            System.out.println("Insufficient funds for withdrawal of " + amount);
        }
    }
}

public class BankAccountSystem {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        Thread t1 = new Thread(() -> account.deposit(500));
        Thread t2 = new Thread(() -> account.withdraw(700));
        Thread t3 = new Thread(() -> account.withdraw(900));

        t1.start();
        t2.start();
        t3.start();
    }
}
```

4. **Multiple orders processed by limited delivery agents (thread pool).**
   Code:

```
package Day9;

import java.util.concurrent.*;

public class OnlineFoodDeliverySystem {
    public static void main(String[] args) {
        ExecutorService deliveryAgents = Executors.newFixedThreadPool(3);

        Runnable orderTask = () -> {
            String orderName = Thread.currentThread().getName();
            System.out.println("Processing order by " + orderName);
            try { Thread.sleep(1000); } catch (InterruptedException e) {
e.printStackTrace(); }
            System.out.println("Order delivered by " + orderName);
        };

        for (int i = 1; i <= 10; i++) {
            deliveryAgents.submit(orderTask);
        }

        deliveryAgents.shutdown();
    }
}
```